

# APS106 Summer 2026

---

WEEK 1 | LECTURE 1

# Introduction

---

# Topics for Today

---

- Introductions
- Set Up and Installations
- Variables, Expressions, and Operators

# Course Motivation

---

- Learn the fundamentals of programming
  - Even as Artificial Intelligence coding abilities improve, we need to understand what it is doing!
- Learn how to apply problem-solving skills in new settings
  - Engineering design through programming

# Why Python?

---

- Simple and clear syntax
- Large open-source community
- The Artificial Intelligence (AI) community overwhelmingly uses Python

# Class Structure

---

- **Lectures** will be used to learn new content and then we will immediately apply said content in practice problems we take up as a class.
- In **tutorials** you will reinforce what you learned in class in a collaborative setting.
- **Lab sessions** are scheduled for you to drop in and ask questions about the weekly assignment or any course content for which you want clarification. **Most lab sessions will take place in GB150 only!**

# Coffee Break / Office Hours

---

- **Availability Form:** <https://www.when2meet.com/?36549103-JDjVn>

# Quercus Run Down

---

| ☰ ▼ Course Information |   |                             | ✓ ▼ | + | ⋮ |
|------------------------|---|-----------------------------|-----|---|---|
| ☰                      | 📄 | Syllabus                    | ✓   |   | ⋮ |
| ☰                      | 📄 | Lecture Homepage            | ✓   |   | ⋮ |
| ☰                      | 📄 | Tutorial Homepage           | ✓   |   | ⋮ |
| ☰                      | 📄 | Lab Homepage                | ✓   |   | ⋮ |
| ☰                      | 📄 | Coffee Break (Office Hours) | ✓   |   | ⋮ |
| ☰                      | 🔗 | Course Anonymous Feedback   | ✓   |   | ⋮ |

# Quercus Run Down

---

| ☰ ▼ Course Resources |   |                            | ✓ ▼ | + | ⋮ |
|----------------------|---|----------------------------|-----|---|---|
| ☰                    | 📄 | How Does That Work? (HDTW) | ✓   |   | ⋮ |
| ☰                    | 📄 | Python Installation        | ✓   |   | ⋮ |
| ☰                    | 📄 | Textbook                   | ✓   |   | ⋮ |
| ☰                    | 📄 | Design Problems            | ✓   |   | ⋮ |
| ☰                    | 📄 | Practice Problems Homepage | ✓   |   | ⋮ |

# Quercus Run Down

|                                    |               |   |   |
|------------------------------------|---------------|---|---|
| ☰ ▼ Week 1   May 4 - 8             | ✓ ▼           | + | ⋮ |
| ☰ 📄 Week 1 Lectures                | ✓             |   | ⋮ |
| ☰ To Do                            | ✓             |   | ⋮ |
| ☰ 📄 Textbook Readings 1            | ✓             |   | ⋮ |
| ☰ 📄 Lab 1<br>May 12   10 pts       | ✓             |   | ⋮ |
| ☰ 🚀 Reflection 1<br>May 10   1 pts | ✓             |   | ⋮ |
| ☰ Tutorial                         | ✓             |   | ⋮ |
| ☰ 📄 Tutorial 1                     | Published ▶ ✓ |   | ⋮ |

# Quercus Run Down

---

## Lecture Homepage

The table below contains the (tentative) schedule for the lectures.

| Week                     | Topics                                                                                                                                                | Material                        | Textbook Readings                   |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|-------------------------------------|
| Week 1 (May 4 - May 8)   | Introductions and Set Up<br>Variables, Expressions, and Operators<br>Functions, Input/Output, Importing Modules<br>Design Problem: Forward Kinematics | <a href="#">Week 1 Lectures</a> | <a href="#">Textbook Readings 1</a> |
| Week 2 (May 11 - May 15) | Conditional Statements + String Comparisons<br>Design Problem: Rock Paper Scissors Lizard Spock!<br>While Loops + Debugging                           | <a href="#">Week 2 Lectures</a> | <a href="#">Textbook Readings 2</a> |
|                          |                                                                                                                                                       |                                 |                                     |

# Grades

---

| Assessment    | Percentage |
|---------------|------------|
| Reflections   | 5%         |
| Labs          | 10%        |
| In-Person Lab | 5%         |
| Midterm       | 30%        |
| Exam          | 50%        |

# Reflections

---

Each week you will complete a reflection that will provide the teaching team up-to-date feedback on what can be improved. In total, these weekly reflections are worth 5% of your final grade. You will receive full marks if you complete provide an answer for every question for every reflection. Please use these reflections to (1) check over your understanding of the material covered that week and (2) tell the teaching team if you have any comments or questions.

# Labs

---

- Four lab assignments (worth 2.5%) will be posted on Quercus weekly, and due about a week later. The labs will be submitted to our auto-marking system Gradescope, and the grades will be entered on Quercus.
- In general: posted Friday at 1:00pm, due the following Friday at 11:00pm.
  - First lab shorter
- No late submissions of the lab assignments will be accepted.

# In-Person Lab

---

The lab assignment in Week 3 will be an in-person quiz worth 5% of your overall grade. The quiz will take place during lab hours on **Friday May 22nd, 2026**. You will use the computers provided in the lab and will only have access to the required files and the material provided on the Quercus page for this course. This lab will also be submitted to our auto-marking system Gradescope at the end of the lab session, and the grades will be entered on Quercus.

# Tips for Success

---

- Do the breakout activities in class!
- Do the weekly practice problems!!
- Ask questions! Feel free to reach out to me any time.

# Set Up

---

# What is Programming/Coding?

---

- A way of telling a computer what to do
  - Computers need every single step written out in a language it understands
  - Runs line-by-line
- How would you request an egg from a human vs. a computer:

- **To a Chef**

- 1. Sunny-side up, please!

- **To a Computer**

- 1. "Turn on stove"
    - 2. "Take out pan"
    - 3. "Take one egg out of fridge"
    - 4. "Crack egg"
    - 5. "Pour egg into pan"
    - 6. "Wait 5 minutes"

# Inside Your Computer

---

- Storage
  - Stores long-term data and is non-volatile
- Memory
  - Random Access Memory (RAM)
  - Stores short-term data and is volatile (loses content when powered off)
  - **Software programs are usually stored and executed from RAM – we will learn how to read from and write to long-term storage later in the term**
- Central Processing Unit (CPU)
  - Runs the programs, including the operating system
  - Reads and writes the data between storage and memory

# Integrated Development Environments (IDEs)

---

- IDEs are programs that provide tools and features to programmers in as unified environment
- Similarly, you use a Word Processor (e.g., Microsoft Word) to write text files
- IDEs include:
  - A code editor (where you type and edit code)
  - Code compilers (translates code for the to understand)
  - Debuggers (allows you to go line-by-line to address errors)
  - A built-in terminal
  - And more

# Python Installation (Anaconda Navigator)

---

- A local distribution system – collection of IDEs, Python packages, and templates for all your Python programming needs
  - Includes Visual Studio (VS) Code, Jupyter Notebooks, PyCharm, and much more
- Advantages:
  - Everything is in one place
  - Comes with everything setup for Python for your PC
  - Can use the built in Anaconda Python distribution with your application of choice
- Download Link: <https://www.anaconda.com/download>

# Visual Studio (VS) Code

---

- Used for .py files
- Not the same as Visual Studio!
- You may use other IDEs if you are comfortable with them

# Jupyter Notebooks

---

- Creates “notebooks” (.ipynb files) that can combine live code, text, and visualizations
- Code can be divided into individually-executable cells
  - Note: this means that the order in which you run these cells matters
- Can export notebooks to HTML or PDF files

# Files Systems

- Organize your files into clear folders! Do not just leave everything in your Downloads folder

[Quit](#)[Logout](#)[Files](#)[Running](#)[Clusters](#)

Select items to perform actions on them.

[Upload](#)[New ▾](#)

| <input type="checkbox"/> 0 ▾ <a href="#">/ Documents / Teaching / APS106 / Lectures / Week 1</a> |  |                 | Name ▾ | Last Modified | File size |
|--------------------------------------------------------------------------------------------------|--|-----------------|--------|---------------|-----------|
| ..                                                                                               |  |                 |        | seconds ago   |           |
| <input type="checkbox"/>                                                                         |  | Lecture 1.ipynb |        | 5 hours ago   | 566 B     |
| <input type="checkbox"/>                                                                         |  | Lecture 2.ipynb |        | seconds ago   | 577 B     |
| <input type="checkbox"/>                                                                         |  | Lecture 1.pptx  |        | 4 minutes ago | 1.17 MB   |
| <input type="checkbox"/>                                                                         |  | Lecture 2.pptx  |        | 4 days ago    | 581 kB    |

# Jupyter Hub

---

- A cloud-based server for running Jupyter notebooks. It does NOT run locally on your computer (like Google Docs compared to Microsoft Word)
- Allows you to 'clone' lecture notebooks and work on notebooks in the cloud
- A good solution if your local Jupyter environment suddenly stops working in lecture

# In This Course

---

- Every lecture and tutorial will use notebook files
- These starter files will be available on Quercus before class begins
- It is recommended to download the files and work on them locally

# Activity!

---

# Variables and Expressions

---

# Comments

---

Comments are text that is ignored when a program is run.

# Comments

---

- When you start a line of code with '#' it is no longer code; it is now a comment that is ignored when the program is run
- The other way to write comments is this:  
'''  
Everything between here is commented out  
This is useful for writing comments over many lines  
We will use this type of comment a lot to explain chunks of code called functions  
'''
- Comments are used to communicate in plain text what code is doing. They are also very useful to help write a plan before solving problems (i.e., pseudo coding)

# The print() Statement

---

The print() statement displays output to the screen.

# The print() Statement

---

- In and of itself does not store information
- Whatever is in the brackets will be the output
- These statements are useful to communicate to code users but also for debugging purposes

# Variables

---

Variables store information (values) in an accessible way.

# Variable Names

---

- Variable names **must**:
  - Start with a letter or an underscore \_
  - Contain only letters, digits, and underscores (no special characters)
- Variable names **should**:
  - Convey meaning about the information it is storing
  - Use the snake case (i.e., separate words with an underscore)

# Assignment Statements

---

First evaluate the thing on the right of =, then assign its address to the thing on the left.

# Assignment Statements

---

- `variable = expression`
- Examples:
  - `X = 20`
  - `X = 10 + 10`

# Memory Visualization Example

```
>>> difference = 20
```

```
>>> double = 2 * difference
```

```
>>> double
```

```
40
```

```
>>> difference = 5
```

```
>>> double
```

```
40
```

1. Evaluate the expression on the right of the = sign -> 20. This produces the value 20, which we'll put at memory address id1.
2. Make the variable on the left of the = sign, difference, refer to 20 by storing id1 in difference.



# Memory Visualization Example

```
>>> difference = 20
```

```
>>> double = 2 * difference
```

```
>>> double
```

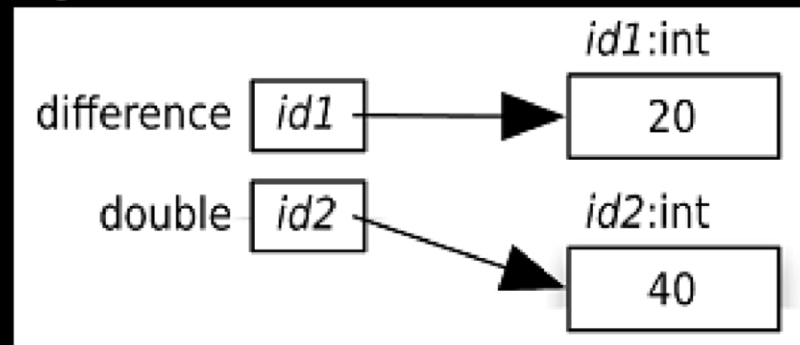
```
40
```

```
>>> difference = 5
```

```
>>> double
```

```
40
```

1. Evaluate the expression on the right of the = sign:  $2 * \text{difference}$ . As we see in the memory model, difference refers to the value 20, so this expression is equivalent to  $2 * 20$ , which produces 40. We'll pick the memory address id2 for the value 40.
2. Make the variable on the left of the = sign, double, refer to 40 by storing id2 in double.



# Memory Visualization Example

---

```
>>> difference = 20
```

```
>>> double = 2 * difference
```

```
>>> double
```

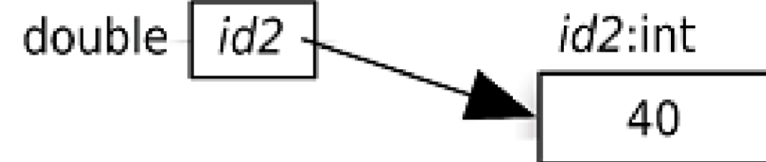
```
40
```

When Python executes the third statement, `double`, it merely looks up the value that `double` refers to (40) and displays it.

```
>>> difference = 5
```

```
>>> double
```

```
40
```



# Memory Visualization Example

```
>>> difference = 20
```

```
>>> double = 2 * difference
```

```
>>> double
```

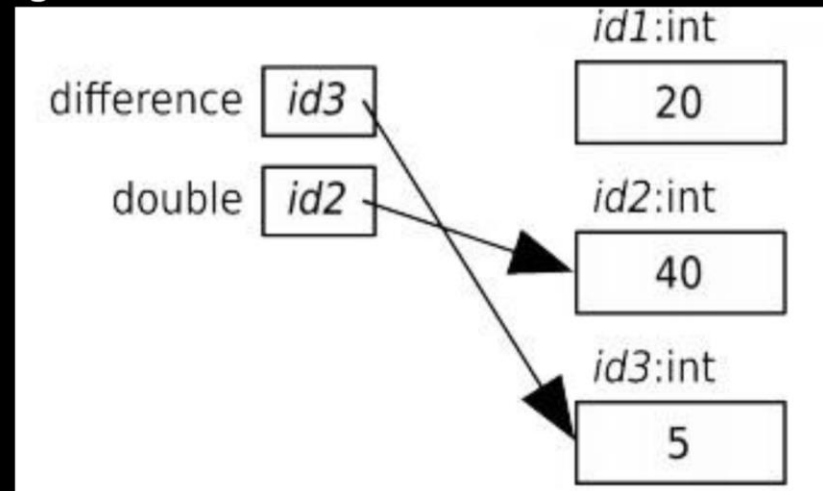
```
40
```

```
>>> difference = 5
```

```
>>> double
```

```
40
```

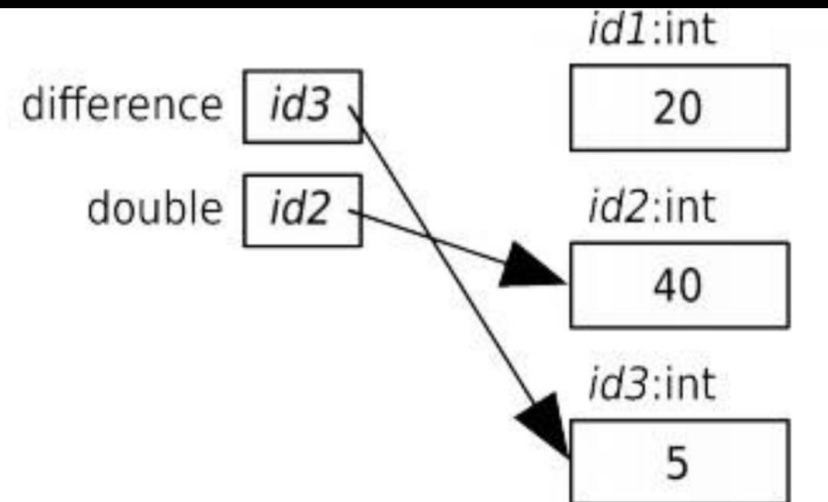
1. Evaluate the expression on the right of the = sign: 5. This produces the value 5, which we'll put at the memory address id3.
2. Make the variable on the left of the = sign, difference, refer to 5 by storing id3 in difference.



# Memory Visualization Example

```
>>> difference = 20
>>> double = 2 * difference
>>> double
40
>>> difference = 5
>>> double
40
```

Variable `double` still contains `id2`, so it still refers to 40. Neither variable refers to 20 anymore which is OK as Python will take care of recycling of memory when it is no longer used.



# Activity!

---

# Variable Types

---

# Variable Types

---

- A type is a set of values and the operations that can be performed on those values.
- You can get the data type of any object by using the `type()` function
- We will learn more types, but the first main three types are...

# Int Type

---

**Int type** represents whole numbers (no decimals)

Ex: 3, 4565490, 10

# Float Type

---

**Float type** represents continuous numbers (decimals)

Ex: 2.5, 1000.23, 4.0

# String Type

---

**String (str)** represents a string of characters (usually text but not always)

Ex: “hello”, ‘a long sentence like this’, “\$#%#^@”, “2.0”

Strings must be surrounded by (matching) quotations

# Activity!

---

# Arithmetic Operators

---

# Arithmetic Operators

---

| Operator | Operation          | Expression | English          | Result |
|----------|--------------------|------------|------------------|--------|
| +        | addition           | 11 + 23    | 11 plus 23       | 34     |
| -        | subtraction        | 23 - 52    | 23 minus 52      | -29    |
| *        | multiplication     | 4 * 5      | 4 times 5        | 20     |
| /        | division           | 9 / 2      | 9 divided by 2   | 4.5    |
| //       | integer division   | 9 // 2     | 9 divided by 2   | 4      |
| **       | exponentiation     | 2 ** 5     | 2 to the power 5 | 32     |
| %        | modulo (remainder) | 9 % 2      | 9 mod 2          | 1      |

# Arithmetic Operator Precedence

---

Liked BEDMAS, when multiple operators are combined in a single expression the operations are evaluated in order of precedence (from left to right)

| Operator                      | Precedence                      |
|-------------------------------|---------------------------------|
| **                            | highest                         |
| - (negation)                  |                                 |
| *, /, //, %                   | (evaluate left-to-right)        |
| + (addition), - (subtraction) | lowest (evaluate left-to-right) |

# Augmented Assignment Operators

---

| Operator         | Expression           | Identical Expression    |
|------------------|----------------------|-------------------------|
| <code>+=</code>  | <code>x += 2</code>  | <code>x = x + 2</code>  |
| <code>-=</code>  | <code>x -= 3</code>  | <code>x = x - 3</code>  |
| <code>*=</code>  | <code>x *= 5</code>  | <code>x = x * 5</code>  |
| <code>/=</code>  | <code>x /= 2</code>  | <code>x = x / 2</code>  |
| <code>//=</code> | <code>x //= 2</code> | <code>x = x // 2</code> |
| <code>**=</code> | <code>x **= 3</code> | <code>x = x ** 3</code> |
| <code>%=</code>  | <code>x %= 3</code>  | <code>x = x % 3</code>  |

# Activity!

---

# Good Practices

---

# Clean Code

---

- **Clean Code** means **Readability**
  - Use whitespace to separate variables and operators
  - Be consistent with spacing, too much whitespace can be bad
  - Pick variable names that are easy to read and interpret
  - Be consistent with naming schemes

# General Guidelines

---

- Use comments!
  - Save yourself from yourself
- Do lots of testing
  - Test often and with purpose
  - Requires you to understand what specific output an input will provide
  - Golden Rule: Never spend more than 15 minutes programming without testing!
- Always have a plan
  - For larger problems we will practice writing steps for ourselves in pseudo code

# Activity!

---